

Machine Learning Using Python

Foundations, Algorithms, Applications, and Reproducible Research

Itauma Itauma

2026-08-20

Table of contents

Preface	18
About This Book	18
Who This Book Is For	18
Second Edition	18
 I Part I — Foundations	 19
1 Machine Learning in the Age of Data and AI	20
1.1 Learning Objectives	20
1.2 Why This Matters	20
1.3 Real-World Problem	20
1.4 Conceptual Foundations	22
1.4.1 Artificial Intelligence, Machine Learning, and Data Science	22
1.4.2 What Does It Mean for a Machine to Learn?	22
1.4.3 Supervised Learning	22
1.4.4 Unsupervised Learning	22
1.4.5 Semi-Supervised and Self-Supervised Learning	22
1.4.6 Reinforcement Learning	22
1.4.7 Deep Learning and Foundation Models	22
1.4.8 Prediction Versus Explanation	22
1.4.9 Machine Learning as an Experimental Process	22
1.5 Mathematical and Statistical Foundations	22
1.5.1 Inputs, Outputs, Features, and Targets	22
1.5.2 Functions and Models	22
1.5.3 Loss and Objective Functions	22
1.5.4 Generalization	22
1.5.5 Bias and Variance	22
1.6 Python Connection	22
1.6.1 The Python Machine Learning Ecosystem	22
1.6.2 A First scikit-learn Workflow	22
1.7 Practitioner Track	22
1.8 Researcher Track	23
1.9 Hands-On Practice	23
1.10 Common Mistakes	23

1.11	Knowledge Check	23
1.12	Real-World Case Study	23
1.12.1	Student Success as a Machine Learning Problem	23
1.13	Professor's Challenge	23
1.14	Research Extension	24
1.15	Reproducibility Check	24
1.16	Key Takeaways	24
2	Python for Machine Learning	25
2.1	Learning Objectives	25
2.2	Why This Matters	25
2.3	Real-World Problem	25
2.4	Conceptual Foundations	26
2.4.1	Python as a Data Science Language	26
2.4.2	Scripts, Notebooks, and Quarto	26
2.4.3	Objects, Functions, and Modules	26
2.4.4	NumPy	26
2.4.5	pandas	26
2.4.6	Visualization	26
2.4.7	scikit-learn	26
2.4.8	Environments and Dependencies	26
2.4.9	Git and GitHub	26
2.5	Python Implementation	26
2.5.1	Inspecting the Environment	26
2.5.2	Loading and Inspecting Data	26
2.5.3	Selecting and Transforming Variables	26
2.5.4	Visualization	26
2.5.5	A Minimal scikit-learn Model	26
2.5.6	Pipelines	26
2.6	Practitioner Track	26
2.7	Researcher Track	27
2.8	Hands-On Practice	27
2.9	Common Mistakes	27
2.10	Knowledge Check	27
2.11	Real-World Case Study	27
2.11.1	Reproducing an Analysis on Another Computer	27
2.12	Professor's Challenge	27
2.13	Research Extension	28
2.14	Reproducibility Check	28
2.15	Key Takeaways	28
3	Statistics and Mathematics for Machine Learning	29
3.1	Learning Objectives	29

3.2	Why This Matters	29
3.3	Real-World Problem	29
3.4	Probability and Statistics	31
3.4.1	Random Variables	31
3.4.2	Probability Distributions	31
3.4.3	Expected Value	31
3.4.4	Variance and Standard Deviation	31
3.4.5	Covariance and Correlation	31
3.4.6	Conditional Probability	31
3.4.7	Bayes' Theorem	31
3.4.8	Sampling and Sampling Variability	31
3.4.9	Confidence Intervals and Hypothesis Tests	31
3.4.10	Association Versus Causation	31
3.5	Linear Algebra	31
3.5.1	Scalars, Vectors, and Matrices	31
3.5.2	Dot Products	31
3.5.3	Matrix Multiplication	31
3.5.4	Transpose and Inverse	31
3.5.5	Eigenvalues and Eigenvectors	31
3.6	Calculus and Optimization	31
3.6.1	Functions and Slopes	31
3.6.2	Derivatives	31
3.6.3	Partial Derivatives	31
3.6.4	Gradients	31
3.6.5	Gradient Descent	31
3.7	Mathematical Connection to Machine Learning	31
3.7.1	Loss Functions	31
3.7.2	Maximum Likelihood	31
3.7.3	Regularization	31
3.8	Hands-On Practice	31
3.9	Common Mistakes	32
3.10	Knowledge Check	32
3.11	Real-World Case Study	32
3.11.1	Understanding Health Risk Variables	32
3.12	Professor's Challenge	32
3.13	Research Extension	32
3.14	Reproducibility Check	32
3.15	Key Takeaways	33

II	Part II — The Machine Learning Workflow	34
4	From Research Question to Machine Learning Problem	35
4.1	Learning Objectives	35
4.2	Why This Matters	35
4.3	Real-World Problem	35
4.4	Problem Formulation	37
4.4.1	Start With the Decision	37
4.4.2	Define the Unit of Analysis	37
4.4.3	Define the Target	37
4.4.4	Define the Prediction Time	37
4.4.5	Identify Available Features	37
4.4.6	Establish a Baseline	37
4.4.7	Choose Evaluation Criteria	37
4.4.8	Consider Consequences of Errors	37
4.5	Prediction, Inference, and Causation	37
4.5.1	Predictive Questions	37
4.5.2	Explanatory Questions	37
4.5.3	Causal Questions	37
4.5.4	Descriptive and Exploratory Questions	37
4.6	Research Design Connection	37
4.6.1	Research Questions	37
4.6.2	Hypotheses	37
4.6.3	Theory and Feature Selection	37
4.6.4	External Validity	37
4.7	Hands-On Practice	37
4.8	Common Mistakes	38
4.9	Knowledge Check	38
4.10	Real-World Case Study	38
4.10.1	Predicting Institutional Retention Outcomes	38
4.11	Professor’s Challenge	38
4.12	Research Extension	38
4.13	Reproducibility Check	38
4.14	Key Takeaways	39
5	Data Preparation and Feature Engineering	40
5.1	Learning Objectives	40
5.2	Why This Matters	40
5.3	Real-World Problem	40
5.4	Data Quality	42
5.4.1	Data Types	42
5.4.2	Missing Data	42
5.4.3	Duplicates	42

5.4.4	Outliers	42
5.4.5	Inconsistent Coding	42
5.5	Feature Engineering	42
5.5.1	Transformations	42
5.5.2	Categorical Encoding	42
5.5.3	Scaling	42
5.5.4	Interaction Features	42
5.5.5	Domain-Informed Features	42
5.5.6	Date and Time Features	42
5.6	Leakage-Free Preprocessing	42
5.6.1	Train-Test Separation	42
5.6.2	ColumnTransformer	42
5.6.3	Pipelines	42
5.6.4	Fit Versus Transform	42
5.7	Python Implementation	42
5.7.1	Numeric and Categorical Pipelines	42
5.7.2	Missing-Value Imputation	42
5.7.3	One-Hot Encoding	42
5.7.4	Standardization	42
5.7.5	Integrated Modeling Pipelines	42
5.8	Hands-On Practice	42
5.9	Common Mistakes	43
5.10	Knowledge Check	43
5.11	Real-World Case Study	43
5.11.1	Preparing Public Education Data for Modeling	43
5.12	Professor’s Challenge	43
5.13	Research Extension	43
5.14	Reproducibility Check	43
5.15	Key Takeaways	44

III Part III — Supervised Learning 45

6 Linear Regression 46

6.1	Learning Objectives	46
6.2	Why This Matters	46
6.3	Real-World Problem	46
6.4	Conceptual Foundations	47
6.4.1	What Is Regression?	47
6.4.2	Prediction Versus Explanation	47
6.4.3	Simple Linear Regression	47
6.4.4	Multiple Linear Regression	47

6.5	Mathematical Foundations	47
6.5.1	The Linear Regression Model	47
6.5.2	Ordinary Least Squares	48
6.5.3	Loss Functions	48
6.6	Assumptions and Statistical Foundations	49
6.6.1	Linearity	49
6.6.2	Independence	49
6.6.3	Constant Variance	49
6.6.4	Normally Distributed Errors	49
6.6.5	Multicollinearity	49
6.6.6	Statistical Inference Versus Predictive Modeling	49
6.7	Python Implementation	49
6.7.1	Required Libraries	49
6.7.2	Loading the Data	51
6.7.3	Exploratory Data Analysis	51
6.7.4	Creating Training and Test Sets	51
6.7.5	Fitting the Model	51
6.7.6	Generating Predictions	51
6.8	Understanding the Output	51
6.8.1	Interpreting the Intercept	51
6.8.2	Interpreting Regression Coefficients	51
6.8.3	Interpreting Prediction Error	51
6.9	Model Diagnostics	51
6.9.1	Residual Analysis	51
6.9.2	Outliers and Influential Observations	51
6.9.3	Multicollinearity	51
6.9.4	Data Leakage	51
6.10	Improving the Model	51
6.10.1	Polynomial Features	51
6.10.2	Ridge Regression	51
6.10.3	Lasso Regression	51
6.10.4	Elastic Net	51
6.11	Evaluating Predictive Performance	51
6.11.1	Training Error Versus Test Error	51
6.11.2	Cross-Validation	51
6.11.3	Benchmark Models	51
6.12	Practitioner Track	51
6.13	Researcher Track	52
6.14	Hands-On Practice	52
6.14.1	Practice 1 — Build a Baseline Model	52
6.14.2	Practice 2 — Compare Models	52
6.15	Common Mistakes	53
6.16	Knowledge Check	53

6.17	Real-World Case Study	53
6.17.1	Predicting a Health Outcome	53
6.18	Professor's Challenge	54
6.19	Research Extension	54
6.20	Reproducibility Check	55
6.21	Key Takeaways	55
7	Classification and Logistic Regression	56
7.1	Learning Objectives	56
7.2	Why This Matters	56
7.3	Real-World Problem	56
7.4	Conceptual Foundations	57
7.4.1	Classification Problems	57
7.4.2	Probabilities, Classes, and Thresholds	57
7.4.3	Logistic Regression	57
7.5	Mathematical Foundations	57
7.5.1	The Logistic Function	57
7.6	Evaluation	58
7.6.1	Confusion Matrix	58
7.6.2	Accuracy	58
7.6.3	Sensitivity and Specificity	58
7.6.4	Precision and Recall	58
7.6.5	F1 Score	58
7.6.6	ROC and AUC	58
7.6.7	Precision-Recall Curves	58
7.6.8	Calibration	58
7.6.9	Threshold Selection	58
7.7	Python Implementation	58
7.7.1	LogisticRegression	58
7.7.2	Predicted Probabilities	58
7.7.3	Classification Metrics	58
7.8	Hands-On Practice	58
7.9	Common Mistakes	58
7.10	Knowledge Check	59
7.11	Real-World Case Study	59
7.11.1	Health Risk Classification	59
7.12	Professor's Challenge	59
7.13	Research Extension	59
7.14	Reproducibility Check	59
7.15	Key Takeaways	59
8	Decision Trees and Random Forests	60
8.1	Learning Objectives	60

8.2	Why This Matters	60
8.3	Real-World Problem	60
8.4	Decision Trees	60
8.4.1	Recursive Partitioning	60
8.4.2	Splits and Leaves	60
8.4.3	Gini Impurity	60
8.5	Random Forests	61
8.5.1	Bootstrap Aggregation	61
8.5.2	Random Feature Selection	61
8.5.3	Out-of-Bag Samples	61
8.5.4	Bias-Variance Intuition	61
8.6	Interpretation	61
8.6.1	Impurity-Based Importance	61
8.6.2	Permutation Importance	61
8.6.3	Limits of Feature Importance	61
8.7	Python Implementation	61
8.7.1	DecisionTreeClassifier and Regressor	61
8.7.2	RandomForestClassifier and Regressor	61
8.8	Hands-On Practice	61
8.9	Common Mistakes	61
8.10	Knowledge Check	62
8.11	Real-World Case Study	62
8.11.1	Predicting an Education Outcome	62
8.12	Professor's Challenge	62
8.13	Research Extension	62
8.14	Reproducibility Check	62
8.15	Key Takeaways	62
9	Support Vector Machines and Kernel Methods	63
9.1	Learning Objectives	63
9.2	Why This Matters	63
9.3	Real-World Problem	63
9.4	Mathematical Foundations	64
9.4.1	Hyperplanes	64
9.4.2	Margins	64
9.4.3	Support Vectors	64
9.4.4	Maximum-Margin Optimization	64
9.5	Python Implementation	64
9.5.1	Scaling	64
9.5.2	SVC	64
9.5.3	Hyperparameter Tuning	64
9.5.4	Probability Estimates	64
9.6	Hands-On Practice	64

9.7	Common Mistakes	64
9.8	Knowledge Check	65
9.9	Real-World Case Study	65
9.9.1	Nonlinear Classification	65
9.10	Professor's Challenge	65
9.11	Research Extension	65
9.12	Reproducibility Check	65
9.13	Key Takeaways	65
10	Ensemble Learning and Gradient Boosting	66
10.1	Learning Objectives	66
10.2	Why This Matters	66
10.3	Real-World Problem	66
10.4	Ensemble Foundations	67
10.4.1	Wisdom of Multiple Models	67
10.4.2	Bagging	67
10.4.3	Boosting	67
10.4.4	Stacking	67
10.5	Gradient Boosting	67
10.5.1	Sequential Error Correction	67
10.5.2	Learning Rate	67
10.5.3	Number of Estimators	67
10.5.4	Tree Depth	67
10.5.5	Regularization	67
10.5.6	XGBoost, LightGBM, and Related Approaches	67
10.6	Python Implementation	67
10.6.1	GradientBoosting	67
10.6.2	HistGradientBoosting	67
10.6.3	Comparing Ensembles	67
10.7	Hands-On Practice	67
10.8	Common Mistakes	68
10.9	Knowledge Check	68
10.10	Real-World Case Study	68
10.10.1	Benchmarking Tabular Models	68
10.11	Professor's Challenge	68
10.12	Research Extension	68
10.13	Reproducibility Check	68
10.14	Key Takeaways	69

IV Part IV — Unsupervised Learning	70
11 Clustering	71
11.1 Learning Objectives	71
11.2 Why This Matters	71
11.3 Real-World Problem	71
11.4 Foundations	72
11.4.1 What Is a Cluster?	72
11.4.2 Distance and Similarity	72
11.4.3 Euclidean Distance	72
11.4.4 Feature Scaling	72
11.5 K-Means	72
11.5.1 Centroids	72
11.5.2 Objective Function	72
11.6 Hierarchical Clustering	73
11.6.1 Linkage	73
11.6.2 Dendrograms	73
11.7 DBSCAN	73
11.7.1 Density	73
11.7.2 Epsilon and Minimum Samples	73
11.7.3 Noise Points	73
11.8 Cluster Evaluation	73
11.8.1 Inertia	73
11.8.2 Silhouette Score	73
11.8.3 Stability	73
11.8.4 External and Substantive Validation	73
11.9 Hands-On Practice	73
11.10 Common Mistakes	73
11.11 Knowledge Check	74
11.12 Real-World Case Study	74
11.12.1 Institutional Profiles	74
11.13 Professor's Challenge	74
11.14 Research Extension	74
11.15 Reproducibility Check	74
11.16 Key Takeaways	74
12 Dimensionality Reduction	75
12.1 Learning Objectives	75
12.2 Why This Matters	75
12.3 Real-World Problem	75
12.4 PCA Foundations	76
12.4.1 Variance and Covariance	76
12.4.2 Eigenvalues and Eigenvectors	76

12.4.3	Principal Components	76
12.4.4	Explained Variance	76
12.4.5	Loadings and Scores	76
12.4.6	Scaling Before PCA	76
12.5	Related Methods	76
12.5.1	Factor Analysis	76
12.5.2	t-SNE	76
12.5.3	UMAP	76
12.5.4	Visualization Versus Modeling	76
12.6	Python Implementation	76
12.6.1	StandardScaler	76
12.6.2	PCA	76
12.6.3	Explained Variance Plots	76
12.6.4	Component Interpretation	76
12.7	Hands-On Practice	76
12.8	Common Mistakes	77
12.9	Knowledge Check	77
12.10	Real-World Case Study	77
12.10.1	Reducing Correlated Institutional Indicators	77
12.11	Professor's Challenge	77
12.12	Research Extension	77
12.13	Reproducibility Check	77
12.14	Key Takeaways	78

V Part V — Modern Machine Learning 79

13 Neural Networks and Deep Learning 80

13.1	Learning Objectives	80
13.2	Why This Matters	80
13.3	Real-World Problem	80
13.4	Mathematical Foundations	80
13.4.1	The Artificial Neuron	80
13.5	Deep Learning Practice	81
13.5.1	Training and Validation	81
13.5.2	Epochs and Batches	81
13.5.3	Learning Rates	81
13.5.4	Regularization	81
13.5.5	Dropout	81
13.5.6	Early Stopping	81
13.6	Python Implementation	81
13.6.1	Building a Small Network	81
13.6.2	Training Curves	81

13.6.3	Evaluating Generalization	81
13.7	Hands-On Practice	81
13.8	Common Mistakes	81
13.9	Knowledge Check	82
13.10	Real-World Case Study	82
13.10.1	Classical ML Versus Neural Networks	82
13.11	Professor's Challenge	82
13.12	Research Extension	82
13.13	Reproducibility Check	82
13.14	Key Takeaways	82
14	Natural Language Processing	83
14.1	Learning Objectives	83
14.2	Why This Matters	83
14.3	Real-World Problem	83
14.4	Text Representation	85
14.4.1	Tokenization	85
14.4.2	Bag of Words	85
14.4.3	TF-IDF	85
14.4.4	N-Grams	85
14.4.5	Word and Document Embeddings	85
14.5	Modeling Text	85
14.5.1	Naive Bayes	85
14.5.2	Logistic Regression	85
14.5.3	Linear SVM	85
14.5.4	Neural Representations	85
14.6	Transformers	85
14.6.1	Attention	85
14.6.2	Contextual Embeddings	85
14.6.3	Pretrained Language Models	85
14.6.4	Fine-Tuning and Retrieval	85
14.7	Evaluation	85
14.7.1	Classification Metrics	85
14.7.2	Data Leakage in Text	85
14.7.3	Domain Shift	85
14.7.4	Human Evaluation	85
14.8	Hands-On Practice	85
14.9	Common Mistakes	86
14.10	Knowledge Check	86
14.11	Real-World Case Study	86
14.11.1	Analyzing Educational Text	86
14.12	Professor's Challenge	86
14.13	Research Extension	86

14.14	Reproducibility Check	86
14.15	Key Takeaways	87
15	Generative AI and Foundation Models	88
15.1	Learning Objectives	88
15.2	Why This Matters	88
15.3	Real-World Problem	88
15.4	Foundations	90
15.4.1	Generative Versus Discriminative Modeling	90
15.4.2	Language Modeling	90
15.4.3	Tokens and Context	90
15.4.4	Transformers and Attention	90
15.4.5	Next-Token Prediction	90
15.4.6	Foundation Models	90
15.5	Working With Foundation Models	90
15.5.1	Prompting	90
15.5.2	Structured Outputs	90
15.5.3	Embeddings	90
15.5.4	Retrieval-Augmented Generation	90
15.5.5	Fine-Tuning	90
15.5.6	Tool Use and Agents	90
15.6	Evaluation	90
15.6.1	Factuality	90
15.6.2	Reliability	90
15.6.3	Human Evaluation	90
15.6.4	Task-Based Evaluation	90
15.6.5	Reproducibility	90
15.6.6	Cost and Latency	90
15.7	Responsible Use	90
15.7.1	Hallucination	90
15.7.2	Bias	90
15.7.3	Privacy	90
15.7.4	Academic Integrity	90
15.7.5	AI-Assisted Coding	90
15.8	Hands-On Practice	90
15.9	Common Mistakes	91
15.10	Knowledge Check	91
15.11	Real-World Case Study	91
15.11.1	AI-Assisted Research Feedback	91
15.12	Professor's Challenge	91
15.13	Research Extension	91
15.14	Reproducibility Check	91
15.15	Key Takeaways	92

VI Part VI — Reliable and Research-Ready Machine Learning	93
16 Model Evaluation, Validation, and Hyperparameter Optimization	94
16.1 Learning Objectives	94
16.2 Why This Matters	94
16.3 Real-World Problem	94
16.4 Evaluation Foundations	96
16.4.1 Generalization Error	96
16.4.2 Train, Validation, and Test Sets	96
16.4.3 K-Fold Cross-Validation	96
16.4.4 Stratification	96
16.4.5 Grouped and Time-Based Splits	96
16.4.6 Repeated Cross-Validation	96
16.4.7 Nested Cross-Validation	96
16.5 Hyperparameter Optimization	96
16.5.1 Grid Search	96
16.5.2 Randomized Search	96
16.5.3 Bayesian Optimization	96
16.5.4 Search Budgets	96
16.6 Comparing Models	96
16.6.1 Baselines	96
16.6.2 Performance Distributions	96
16.6.3 Confidence Intervals	96
16.6.4 Practical Versus Statistical Differences	96
16.6.5 External Validation	96
16.7 Learning Curves	96
16.7.1 Diagnosing Bias	96
16.7.2 Diagnosing Variance	96
16.8 Hands-On Practice	96
16.9 Common Mistakes	97
16.10 Knowledge Check	97
16.11 Real-World Case Study	97
16.11.1 A Reproducible Model Benchmark	97
16.12 Professor’s Challenge	97
16.13 Research Extension	97
16.14 Reproducibility Check	97
16.15 Key Takeaways	98
17 Explainable, Fair, and Responsible Machine Learning	99
17.1 Learning Objectives	99
17.2 Why This Matters	99
17.3 Real-World Problem	99

17.4	Explainability	101
17.4.1	Model Transparency	101
17.4.2	Global Versus Local Explanations	101
17.4.3	Permutation Importance	101
17.4.4	Partial Dependence	101
17.4.5	SHAP	101
17.4.6	Limits of Post-Hoc Explanation	101
17.5	Calibration and Reliability	101
17.5.1	Predicted Probability Versus Observed Frequency	101
17.5.2	Calibration Curves	101
17.5.3	Brier Score	101
17.6	Fairness	101
17.6.1	Subgroup Performance	101
17.6.2	Demographic Parity	101
17.6.3	Equal Opportunity	101
17.6.4	Equalized Odds	101
17.6.5	Calibration Across Groups	101
17.6.6	Fairness Tradeoffs	101
17.7	Responsible ML	101
17.7.1	Intended Use	101
17.7.2	Misuse	101
17.7.3	Human Oversight	101
17.7.4	Documentation	101
17.7.5	Model Cards	101
17.7.6	Monitoring and Drift	101
17.8	Hands-On Practice	101
17.9	Common Mistakes	102
17.10	Knowledge Check	102
17.11	Real-World Case Study	102
17.11.1	Responsible Prediction in Health or Education	102
17.12	Professor's Challenge	102
17.13	Research Extension	102
17.14	Reproducibility Check	102
17.15	Key Takeaways	103
18	Reproducible Machine Learning and Research Communication	104
18.1	Learning Objectives	104
18.2	Why This Matters	104
18.3	Real-World Problem	104
18.4	Reproducible Project Design	106
18.4.1	Project Structure	106
18.4.2	Raw Versus Derived Data	106
18.4.3	Scripts and Notebooks	106

18.4.4	Quarto	106
18.4.5	Git and GitHub	106
18.4.6	Environments and Dependencies	106
18.4.7	Randomness and Seeds	106
18.5	Data Provenance	106
18.5.1	Data Sources	106
18.5.2	Data Dictionaries	106
18.5.3	Licensing	106
18.5.4	Versioning	106
18.5.5	Sensitive Data	106
18.6	Model Provenance	106
18.6.1	Training Configuration	106
18.6.2	Hyperparameters	106
18.6.3	Evaluation Protocol	106
18.6.4	Artifacts and Model Cards	106
18.7	Scholarly Communication	106
18.7.1	Tables and Figures	106
18.7.2	Methods Reporting	106
18.7.3	Limitations	106
18.7.4	Repositories	106
18.7.5	CITATION.cff	106
18.7.6	Releases	106
18.7.7	Archiving and DOIs	106
18.8	From Class Project to Research Product	106
18.8.1	Replication	106
18.8.2	Extension	106
18.8.3	Benchmarking	106
18.8.4	Methodological Contribution	106
18.8.5	Substantive Contribution	106
18.9	Hands-On Practice	106
18.10	Common Mistakes	107
18.11	Knowledge Check	107
18.12	Real-World Case Study	107
18.12.1	From Notebook to Citable Research Artifact	107
18.13	Professor’s Challenge	107
18.14	Research Extension	107
18.15	Reproducibility Check	107
18.16	Key Takeaways	108

References	109
-------------------	------------

Preface

About This Book

Machine Learning Using Python: Foundations, Algorithms, Applications, and Reproducible Research is designed for data enthusiasts, analysts, students, researchers, and professionals who want to develop a deeper understanding of machine learning using Python.

This Second Edition integrates conceptual foundations, mathematical reasoning, practical programming, real-world applications, responsible machine learning, and reproducible research.

Who This Book Is For

This book is intended for readers who want to move beyond simply fitting machine learning models toward understanding how algorithms work, how models should be evaluated, and how machine learning can be applied responsibly to meaningful real-world problems.

Second Edition

This edition substantially expands the first edition with greater emphasis on mathematical foundations, modern machine learning methods, hands-on practice, health and education applications, research design, reproducibility, and responsible artificial intelligence.

Part I

Part I — Foundations

1 Machine Learning in the Age of Data and AI

1.1 Learning Objectives

- Define machine learning and distinguish it from related areas of artificial intelligence.
- Differentiate supervised, unsupervised, semi-supervised, and reinforcement learning.
- Distinguish prediction, explanation, inference, and decision-making objectives.
- Describe the end-to-end machine learning lifecycle.
- Identify opportunities and limitations of machine learning in real-world settings.

1.2 Why This Matters

Machine learning now supports decisions across health, education, finance, science, business, and public policy. Effective data scientists need more than the ability to call an algorithm: they must understand what problem is being solved, what evidence a model provides, and when machine learning is inappropriate.

1.3 Real-World Problem

Consider a university seeking to identify students who may need additional academic support. What can a predictive model legitimately tell decision-makers, and what can it not tell them?

1.4 Conceptual Foundations

1.4.1 Artificial Intelligence, Machine Learning, and Data Science

1.4.2 What Does It Mean for a Machine to Learn?

1.4.3 Supervised Learning

1.4.4 Unsupervised Learning

1.4.5 Semi-Supervised and Self-Supervised Learning

1.4.6 Reinforcement Learning

1.4.7 Deep Learning and Foundation Models

1.4.8 Prediction Versus Explanation

1.4.9 Machine Learning as an Experimental Process

1.5 Mathematical and Statistical Foundations

1.5.1 Inputs, Outputs, Features, and Targets

1.5.2 Functions and Models

1.5.3 Loss and Objective Functions

1.5.4 Generalization

1.5.5 Bias and Variance

1.6 Python Connection

1.6.1 The Python Machine Learning Ecosystem

1.6.2 A First scikit-learn Workflow

1.7 Practitioner Track

Focus on recognizing ML problem types, selecting appropriate workflows, and interpreting model outputs.

1.8 Researcher Track

Focus additionally on research questions, generalization, benchmarking, uncertainty, reproducibility, and scientific contribution.

1.9 Hands-On Practice

Classify a collection of real-world problems by learning type, target, unit of analysis, and appropriate evaluation strategy.

1.10 Common Mistakes

- Treating every data problem as a machine learning problem.
- Confusing prediction with causation.
- Choosing an algorithm before defining the problem.
- Reporting accuracy without considering the decision context.

1.11 Knowledge Check

1. How does supervised learning differ from unsupervised learning?
2. What is generalization?
3. Why does prediction not establish causation?
4. When might a simple model be preferable to a complex model?

1.12 Real-World Case Study

1.12.1 Student Success as a Machine Learning Problem

Formulate a student-success problem using a public education dataset and identify the target, predictors, unit of analysis, risks, and evaluation strategy.

1.13 Professor's Challenge

A model predicts student non-retention with 90% accuracy. Is that enough evidence to deploy it? Explain what else you would need to know.

1.14 Research Extension

Identify a substantive gap that could transform the case study from a classroom prediction exercise into a research contribution.

1.15 Reproducibility Check

Document the problem definition, data source, target definition, assumptions, and intended use before modeling begins.

1.16 Key Takeaways

- Machine learning is a problem-solving framework, not merely a collection of algorithms.
- Prediction, inference, and causation answer different questions.
- Generalization to unseen data is central to machine learning.
- Responsible modeling begins before code is written.

2 Python for Machine Learning

2.1 Learning Objectives

- Configure a reproducible Python environment for machine learning.
- Work with NumPy arrays and pandas DataFrames.
- Visualize data for exploratory analysis.
- Use scikit-learn's estimator and pipeline APIs.
- Organize machine learning code for reproducibility.

2.2 Why This Matters

Reliable machine learning depends on reliable computational workflows. Python provides a mature ecosystem for data manipulation, visualization, statistical modeling, machine learning, and deep learning.

2.3 Real-World Problem

A colleague sends you a notebook that worked six months ago but no longer runs. How can environments, dependency management, version control, and reproducible code prevent this problem?

2.4 Conceptual Foundations

2.4.1 Python as a Data Science Language

2.4.2 Scripts, Notebooks, and Quarto

2.4.3 Objects, Functions, and Modules

2.4.4 NumPy

2.4.5 pandas

2.4.6 Visualization

2.4.7 scikit-learn

2.4.8 Environments and Dependencies

2.4.9 Git and GitHub

2.5 Python Implementation

2.5.1 Inspecting the Environment

2.5.2 Loading and Inspecting Data

2.5.3 Selecting and Transforming Variables

2.5.4 Visualization

2.5.5 A Minimal scikit-learn Model

2.5.6 Pipelines

2.6 Practitioner Track

Develop fluency with the core tools needed to build ML workflows.

2.7 Researcher Track

Emphasize computational provenance, environment capture, deterministic execution, and auditable analysis.

2.8 Hands-On Practice

Create a small reproducible analysis that loads a public dataset, summarizes it, visualizes key variables, and fits a baseline model.

2.9 Common Mistakes

- Installing packages into uncontrolled environments.
- Hard-coding local file paths.
- Mixing data cleaning and modeling without documentation.
- Failing to record package versions.

2.10 Knowledge Check

1. What problem does a virtual environment solve?
2. Why are pipelines useful?
3. How do NumPy and pandas differ?
4. Why should file paths be portable?

2.11 Real-World Case Study

2.11.1 Reproducing an Analysis on Another Computer

Package an analysis so that another researcher can clone the repository and reproduce the results.

2.12 Professor's Challenge

Your model gives different results on two computers. Develop a systematic debugging plan.

2.13 Research Extension

Evaluate reproducibility practices in a published open-source machine learning project.

2.14 Reproducibility Check

Verify relative paths, dependencies, seeds, data provenance, and a clean end-to-end render.

2.15 Key Takeaways

- Reproducibility is part of technical competence.
- Python's ML ecosystem is strongest when tools are integrated into a disciplined workflow.
- Portable code reduces dependence on individual analysts.

3 Statistics and Mathematics for Machine Learning

3.1 Learning Objectives

- Explain the statistical concepts most frequently used in machine learning.
- Work with vectors, matrices, probabilities, expectations, and variance.
- Interpret covariance and correlation.
- Explain derivatives, gradients, and optimization conceptually.
- Connect probability and linear algebra to ML algorithms.

3.2 Why This Matters

Machine learning algorithms encode mathematical assumptions. Understanding those assumptions makes it easier to diagnose models, choose methods, and interpret results.

3.3 Real-World Problem

Two predictors are strongly correlated and a model's coefficients become unstable. What mathematical and statistical concepts explain the problem?

3.4 Probability and Statistics

3.4.1 Random Variables

3.4.2 Probability Distributions

3.4.3 Expected Value

3.4.4 Variance and Standard Deviation

3.4.5 Covariance and Correlation

3.4.6 Conditional Probability

3.4.7 Bayes' Theorem

3.4.8 Sampling and Sampling Variability

3.4.9 Confidence Intervals and Hypothesis Tests

3.4.10 Association Versus Causation

3.5 Linear Algebra

3.5.1 Scalars, Vectors, and Matrices

3.5.2 Dot Products

3.5.3 Matrix Multiplication

3.5.4 Transpose and Inverse

3.5.5 Eigenvalues and Eigenvectors

3.6 Calculus and Optimization

3.6.1 Functions and Slopes

3.6.2 Derivatives

3.6.3 Partial Derivatives

3.6.4 Gradients

3.6.5 Gradient Descent

3.7 Mathematical Connection to Machine Learning

3.7.1 Loss Functions

3.9 Common Mistakes

- Treating correlation as causation.
- Assuming statistical significance implies predictive usefulness.
- Ignoring scale when distance is important.
- Applying formulas without understanding assumptions.

3.10 Knowledge Check

1. What does covariance measure?
2. How is correlation different from covariance?
3. What does a gradient represent?
4. Why are eigenvectors important in PCA?
5. What is conditional probability?

3.11 Real-World Case Study

3.11.1 Understanding Health Risk Variables

Explore relationships among demographic, behavioral, and biometric variables in a public health dataset.

3.12 Professor's Challenge

A variable is statistically significant but adds almost no out-of-sample predictive performance. Should it remain in a predictive model?

3.13 Research Extension

Formulate a study comparing inferential and predictive conclusions from the same dataset.

3.14 Reproducibility Check

Record transformations, statistical assumptions, and all computational choices.

3.15 Key Takeaways

- Statistics, linear algebra, probability, and optimization form the language of machine learning.
- Mathematical understanding improves model selection and diagnosis.
- Predictive and inferential usefulness are not identical.

Part II

Part II — The Machine Learning Workflow

4 From Research Question to Machine Learning Problem

4.1 Learning Objectives

- Translate substantive questions into well-defined ML tasks.
- Define units of analysis, targets, features, and prediction horizons.
- Distinguish predictive, inferential, descriptive, and causal questions.
- Develop defensible baseline and evaluation strategies.
- Identify leakage and target-definition problems before modeling.

4.2 Why This Matters

Many failed ML projects begin with an algorithm instead of a clearly defined problem.

4.3 Real-World Problem

A university asks: “Which students will struggle?” Convert this vague request into a precise, ethical, and evaluable machine learning problem.

4.4 Problem Formulation

4.4.1 Start With the Decision

4.4.2 Define the Unit of Analysis

4.4.3 Define the Target

4.4.4 Define the Prediction Time

4.4.5 Identify Available Features

4.4.6 Establish a Baseline

4.4.7 Choose Evaluation Criteria

4.4.8 Consider Consequences of Errors

4.5 Prediction, Inference, and Causation

4.5.1 Predictive Questions

4.5.2 Explanatory Questions

4.5.3 Causal Questions

4.5.4 Descriptive and Exploratory Questions

4.6 Research Design Connection

4.6.1 Research Questions

4.6.2 Hypotheses

4.6.3 Theory and Feature Selection

4.6.4 External Validity

4.7 Hands-On Practice

Rewrite five vague analytics requests as formal ML problem statements.

4.8 Common Mistakes

- Predicting information that is already known at prediction time.
- Using future information as a feature.
- Defining targets inconsistently.
- Optimizing a metric unrelated to the actual decision.

4.9 Knowledge Check

1. What is the unit of analysis?
2. What is a prediction horizon?
3. How can target leakage occur?
4. Why should a baseline be established before complex modeling?

4.10 Real-World Case Study

4.10.1 Predicting Institutional Retention Outcomes

Formulate an education analytics problem using public institutional data.

4.11 Professor's Challenge

A feature dramatically improves performance but is only observed after the outcome occurs. Can it be used?

4.12 Research Extension

Develop a research question and contribution statement around the case study.

4.13 Reproducibility Check

Version the problem statement and document target, features, exclusions, timing, and evaluation plan.

4.14 Key Takeaways

- Problem formulation precedes algorithm selection.
- Time and data availability determine whether a prediction is legitimate.
- Evaluation metrics must reflect the decision context.

5 Data Preparation and Feature Engineering

5.1 Learning Objectives

- Diagnose data-quality problems.
- Handle missing, categorical, and scaled features appropriately.
- Explain data leakage and prevent it with pipelines.
- Engineer features using substantive knowledge.
- Build preprocessing workflows with scikit-learn.

5.2 Why This Matters

Model quality is constrained by data quality, and seemingly harmless preprocessing can invalidate an evaluation if information leaks across data partitions.

5.3 Real-World Problem

A preprocessing workflow standardizes the entire dataset before the train-test split. Why is the resulting performance estimate potentially optimistic?

5.4 Data Quality

5.4.1 Data Types

5.4.2 Missing Data

5.4.3 Duplicates

5.4.4 Outliers

5.4.5 Inconsistent Coding

5.5 Feature Engineering

5.5.1 Transformations

5.5.2 Categorical Encoding

5.5.3 Scaling

5.5.4 Interaction Features

5.5.5 Domain-Informed Features

5.5.6 Date and Time Features

5.6 Leakage-Free Preprocessing

5.6.1 Train-Test Separation

5.6.2 ColumnTransformer

5.6.3 Pipelines

5.6.4 Fit Versus Transform

5.7 Python Implementation

5.7.1 Numeric and Categorical Pipelines

5.7.2 Missing-Value Imputation

5.7.3 One-Hot Encoding

42

5.7.4 Standardization

5.7.5 Integrated Modeling Pipelines

5.8 Hands-On Practice

5.9 Common Mistakes

- Preprocessing before splitting.
- Imputing using information from the test set.
- Encoding identifiers as meaningful numeric features.
- Feature engineering using future information.

5.10 Knowledge Check

1. Why can scaling cause leakage?
2. When is one-hot encoding appropriate?
3. What does a Pipeline protect against?
4. Why should identifiers usually be excluded?

5.11 Real-World Case Study

5.11.1 Preparing Public Education Data for Modeling

Develop a documented feature matrix from an institutional dataset.

5.12 Professor's Challenge

Two analysts obtain different model results from the same raw data. How would you audit their preprocessing pipelines?

5.13 Research Extension

Compare alternative missing-data or feature-engineering strategies using nested evaluation.

5.14 Reproducibility Check

Preserve raw data, script transformations, document exclusions, and keep preprocessing inside the modeling workflow.

5.15 Key Takeaways

- Preprocessing is part of the model.
- Leakage can invalidate otherwise sophisticated analyses.
- Feature engineering should combine technical and domain knowledge.

Part III

Part III — Supervised Learning

6 Linear Regression

6.1 Learning Objectives

By the end of this chapter, you should be able to:

- Explain the role of linear regression in predictive modeling.
- Distinguish simple linear regression from multiple linear regression.
- Interpret regression coefficients in context.
- Explain the assumptions underlying ordinary least squares regression.
- Derive and interpret common regression evaluation metrics.
- Build a reproducible regression workflow in Python.
- Diagnose common modeling problems such as nonlinearity, multicollinearity, heteroscedasticity, and influential observations.
- Compare predictive performance across alternative regression models.
- Connect regression modeling to real-world research questions.

6.2 Why This Matters

Linear regression is one of the foundational algorithms in statistics and machine learning. Although modern machine learning includes highly complex models, linear regression remains valuable because it is interpretable, computationally efficient, and useful as a benchmark against which more complex models can be evaluated.

6.3 Real-World Problem

Suppose a public health researcher wants to predict systolic blood pressure using demographic, behavioral, and anthropometric characteristics.

Questions might include:

- Which variables are most strongly associated with systolic blood pressure?
- How accurately can blood pressure be predicted?
- Does a more complex model substantially outperform linear regression?

- Are prediction errors larger for some population subgroups than others?

Throughout this chapter, we will use problems like this to connect mathematical foundations with machine learning practice.

6.4 Conceptual Foundations

6.4.1 What Is Regression?

6.4.2 Prediction Versus Explanation

6.4.3 Simple Linear Regression

6.4.4 Multiple Linear Regression

6.5 Mathematical Foundations

6.5.1 The Linear Regression Model

For a single predictor:

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i$$

For multiple predictors:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} + \epsilon_i$$

where:

- y_i is the observed outcome,
- β_0 is the intercept,
- β_j represents the effect associated with predictor j ,
- x_{ij} is the value of predictor j for observation i ,
- ϵ_i represents unexplained variation.

6.5.2 Ordinary Least Squares

The ordinary least squares estimator minimizes the residual sum of squares:

$$RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

In matrix notation:

$$\hat{\beta} = (X^\top X)^{-1} X^\top y$$

6.5.3 Loss Functions

6.5.3.1 Mean Squared Error

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

6.5.3.2 Root Mean Squared Error

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

6.5.3.3 Mean Absolute Error

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

6.5.3.4 Coefficient of Determination

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

6.6 Assumptions and Statistical Foundations

6.6.1 Linearity

6.6.2 Independence

6.6.3 Constant Variance

6.6.4 Normally Distributed Errors

6.6.5 Multicollinearity

6.6.6 Statistical Inference Versus Predictive Modeling

6.7 Python Implementation

6.7.1 Required Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```


6.7.2 Loading the Data

6.7.3 Exploratory Data Analysis

6.7.4 Creating Training and Test Sets

6.7.5 Fitting the Model

6.7.6 Generating Predictions

6.8 Understanding the Output

6.8.1 Interpreting the Intercept

6.8.2 Interpreting Regression Coefficients

6.8.3 Interpreting Prediction Error

6.9 Model Diagnostics

6.9.1 Residual Analysis

6.9.2 Outliers and Influential Observations

6.9.3 Multicollinearity

6.9.4 Data Leakage

6.10 Improving the Model

6.10.1 Polynomial Features

6.10.2 Ridge Regression

6.10.3 Lasso Regression

6.10.4 Elastic Net

6.11 Evaluating Predictive Performance

6.11.1 Training Error Versus Test Error

6.11.2 Cross-Validation

51

6.11.3 Benchmark Models

6.12 Practitioner Track

- fitting the model,
- generating predictions,
- evaluating test-set performance,
- building reproducible pipelines,
- comparing linear regression with stronger predictive models.

6.13 Researcher Track

Focus additionally on:

- assumptions,
- sampling,
- coefficient uncertainty,
- variable selection,
- model specification,
- subgroup performance,
- interpretation of findings,
- reproducibility.

6.14 Hands-On Practice

6.14.1 Practice 1 — Build a Baseline Model

Fit a linear regression model using a real dataset and report:

- MAE,
- RMSE,
- R^2 ,
- residual behavior,
- the strongest predictors.

6.14.2 Practice 2 — Compare Models

Compare:

1. ordinary least squares,
2. Ridge regression,
3. Lasso regression.

Use cross-validation to determine whether regularization improves predictive performance.

6.15 Common Mistakes

- Evaluating the model only on training data.
- Interpreting association as causation.
- Ignoring nonlinearity.
- Performing preprocessing before separating training and test data.
- Using R^2 as the only evaluation metric.
- Removing outliers without substantive justification.
- Selecting predictors solely because they are statistically significant.

6.16 Knowledge Check

1. What does the slope coefficient represent in a simple linear regression model?
2. Why can training-set performance provide an overly optimistic assessment of a model?
3. What is the difference between RMSE and MAE?
4. What does an R^2 value of 0.70 mean?
5. Why can multicollinearity make coefficient interpretation difficult?
6. Why does strong predictive performance not establish causation?

6.17 Real-World Case Study

6.17.1 Predicting a Health Outcome

Develop a regression model using a publicly available health dataset.

Your analysis should include:

1. a clearly stated prediction problem,
2. data preparation,
3. exploratory analysis,
4. model development,
5. test-set evaluation,
6. interpretation,
7. diagnostics,
8. reproducibility documentation.

6.18 Professor's Challenge

A linear regression model produces:

$$R^2 = 0.81$$

on the training data but:

$$R^2 = 0.54$$

on the test data.

A colleague claims:

“The model explains 81% of the variation, so it is an excellent model.”

Critically evaluate this claim.

What additional evidence would you require before recommending the model for decision-making?

6.19 Research Extension

Transform the chapter case study into a potential research project.

Consider:

- What is the substantive research gap?
- What population is being studied?
- Which predictors are theoretically justified?
- What alternative algorithms should be benchmarked?
- How will predictive uncertainty be evaluated?
- Will model performance differ across demographic groups?
- What contribution could make the study publishable?

6.20 Reproducibility Check

Before completing this chapter, verify that:

- the analysis runs from beginning to end,
- random seeds are documented,
- data sources are cited,
- preprocessing occurs inside the modeling workflow where appropriate,
- the test set has not influenced model development,
- software dependencies are documented.

6.21 Key Takeaways

- Linear regression remains an important baseline machine learning algorithm.
- Prediction and statistical inference are related but distinct objectives.
- Model evaluation should focus on unseen data.
- Mathematical understanding helps identify when a model is inappropriate.
- Reproducible workflows are essential for credible machine learning research.

7 Classification and Logistic Regression

7.1 Learning Objectives

- Formulate binary and multiclass classification problems.
- Explain logistic regression mathematically.
- Interpret predicted probabilities and odds.
- Evaluate classifiers using threshold-dependent and threshold-independent metrics.
- Select decision thresholds based on real-world costs.

7.2 Why This Matters

Classification models frequently influence high-stakes decisions. Accuracy alone rarely captures whether those decisions are useful or safe.

7.3 Real-World Problem

A health screening model identifies individuals at elevated risk. Which is more consequential: a false positive or a false negative?

7.4 Conceptual Foundations

7.4.1 Classification Problems

7.4.2 Probabilities, Classes, and Thresholds

7.4.3 Logistic Regression

7.5 Mathematical Foundations

7.5.1 The Logistic Function

$$P(Y = 1|X) = \frac{1}{1 + e^{-z}}$$

Log-Odds

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$$

Maximum Likelihood and Log Loss

7.6 Evaluation

7.6.1 Confusion Matrix

7.6.2 Accuracy

7.6.3 Sensitivity and Specificity

7.6.4 Precision and Recall

7.6.5 F1 Score

7.6.6 ROC and AUC

7.6.7 Precision-Recall Curves

7.6.8 Calibration

7.6.9 Threshold Selection

7.7 Python Implementation

7.7.1 LogisticRegression

7.7.2 Predicted Probabilities

7.7.3 Classification Metrics

7.8 Hands-On Practice

Build and evaluate a binary classifier using a public health dataset.

7.9 Common Mistakes

- Using accuracy with severe class imbalance.
- Treating 0.50 as an automatically optimal threshold.
- Interpreting predicted probabilities as causal effects.
- Evaluating on data used for model selection.

7.10 Knowledge Check

1. What does logistic regression model?
2. How do sensitivity and specificity differ?
3. When is PR-AUC especially useful?
4. Why does threshold choice depend on context?

7.11 Real-World Case Study

7.11.1 Health Risk Classification

Compare logistic regression with a nonlinear classifier and assess discrimination and calibration.

7.12 Professor's Challenge

Model A has higher ROC-AUC, but Model B is better calibrated and identifies more true cases at the operational threshold. Which model should be deployed?

7.13 Research Extension

Investigate subgroup performance or temporal validation as a publishable extension.

7.14 Reproducibility Check

Document class definitions, prevalence, splits, thresholds, metrics, and calibration methods.

7.15 Key Takeaways

- Classification is fundamentally about uncertainty and decisions.
- No single metric is sufficient for every classification problem.
- Thresholds should reflect consequences, not convenience.

8 Decision Trees and Random Forests

8.1 Learning Objectives

- Explain recursive partitioning.
- Calculate and interpret impurity measures.
- Control tree complexity.
- Explain bagging and random forests.
- Evaluate feature importance cautiously.

8.2 Why This Matters

Tree-based models capture nonlinearities and interactions with relatively little preprocessing and form the foundation of many powerful ensemble methods.

8.3 Real-World Problem

Can nonlinear combinations of demographic and behavioral variables improve prediction beyond a linear baseline?

8.4 Decision Trees

8.4.1 Recursive Partitioning

8.4.2 Splits and Leaves

8.4.3 Gini Impurity

$$G = 1 - \sum_k p_k^2$$

Entropy

$$H = - \sum_k p_k \log_2(p_k)$$

Tree Depth and Pruning

8.5 Random Forests

8.5.1 Bootstrap Aggregation

8.5.2 Random Feature Selection

8.5.3 Out-of-Bag Samples

8.5.4 Bias-Variance Intuition

8.6 Interpretation

8.6.1 Impurity-Based Importance

8.6.2 Permutation Importance

8.6.3 Limits of Feature Importance

8.7 Python Implementation

8.7.1 DecisionTreeClassifier and Regressor

8.7.2 RandomForestClassifier and Regressor

8.8 Hands-On Practice

Compare a decision tree and random forest with a linear baseline.

8.9 Common Mistakes

- Allowing unrestricted tree growth.
- Treating feature importance as causality.
- Ignoring correlated predictors.
- Comparing models on different splits.

8.10 Knowledge Check

1. What does Gini impurity measure?
2. Why do random forests decorrelate trees?
3. What is bagging?
4. Why can feature importance be misleading?

8.11 Real-World Case Study

8.11.1 Predicting an Education Outcome

Compare interpretable and ensemble tree models using public education data.

8.12 Professor's Challenge

A random forest outperforms logistic regression by one percentage point but is much harder to explain. Which model should be used?

8.13 Research Extension

Test whether performance differences persist under repeated or external validation.

8.14 Reproducibility Check

Record seeds, split rules, hyperparameters, feature definitions, and evaluation protocol.

8.15 Key Takeaways

- Trees model nonlinear decision boundaries naturally.
- Ensembles reduce instability but increase complexity.
- Importance measures require careful interpretation.

9 Support Vector Machines and Kernel Methods

9.1 Learning Objectives

- Explain margins and support vectors.
- Describe hard- and soft-margin classification.
- Explain the kernel trick conceptually.
- Tune SVM hyperparameters using validation.
- Recognize when feature scaling is essential.

9.2 Why This Matters

Support vector machines illustrate how geometry, optimization, and nonlinear feature spaces combine to produce powerful predictive models.

9.3 Real-World Problem

Can a nonlinear boundary separate complex patient or student profiles better than a linear classifier?

9.4 Mathematical Foundations

9.4.1 Hyperplanes

9.4.2 Margins

9.4.3 Support Vectors

9.4.4 Maximum-Margin Optimization

$$\min_{w,b} \frac{1}{2} \|w\|^2$$

subject to

$$y_i(w^T x_i + b) \geq 1$$

Soft Margins ### Kernels ### Linear, Polynomial, and RBF Kernels

9.5 Python Implementation

9.5.1 Scaling

9.5.2 SVC

9.5.3 Hyperparameter Tuning

9.5.4 Probability Estimates

9.6 Hands-On Practice

Compare linear and RBF SVMs using cross-validation.

9.7 Common Mistakes

- Failing to scale features.
- Tuning on the test set.
- Using complex kernels without justification.
- Treating SVM scores as calibrated probabilities.

9.8 Knowledge Check

1. What is a support vector?
2. What does the margin represent?
3. Why is scaling important?
4. What does the kernel trick accomplish?

9.9 Real-World Case Study

9.9.1 Nonlinear Classification

Benchmark SVM against logistic regression and a tree-based method.

9.10 Professor's Challenge

An RBF SVM wins cross-validation by a small margin but requires much more computation. Is the gain meaningful?

9.11 Research Extension

Study performance, calibration, and computational tradeoffs across kernels.

9.12 Reproducibility Check

Document scaling, search spaces, validation folds, seeds, and final untouched test evaluation.

9.13 Key Takeaways

- SVMs connect geometry and optimization.
- Kernel methods enable nonlinear decision boundaries.
- Proper scaling and validation are essential.

10 Ensemble Learning and Gradient Boosting

10.1 Learning Objectives

- Explain why combining models can improve prediction.
- Distinguish bagging, boosting, and stacking.
- Describe gradient boosting conceptually.
- Tune ensemble models responsibly.
- Compare predictive gains with complexity and interpretability costs.

10.2 Why This Matters

Ensemble methods are among the strongest general-purpose approaches for structured/tabular data.

10.3 Real-World Problem

A baseline model performs reasonably well, but stakeholders want better predictions. How much complexity is justified for an incremental performance gain?

10.4 Ensemble Foundations

10.4.1 Wisdom of Multiple Models

10.4.2 Bagging

10.4.3 Boosting

10.4.4 Stacking

10.5 Gradient Boosting

10.5.1 Sequential Error Correction

10.5.2 Learning Rate

10.5.3 Number of Estimators

10.5.4 Tree Depth

10.5.5 Regularization

10.5.6 XGBoost, LightGBM, and Related Approaches

10.6 Python Implementation

10.6.1 GradientBoosting

10.6.2 HistGradientBoosting

10.6.3 Comparing Ensembles

10.7 Hands-On Practice

Benchmark linear, random forest, and gradient-boosting models using identical validation folds.

10.8 Common Mistakes

- Comparing models under different evaluation protocols.
- Excessive hyperparameter searching.
- Reporting only the best result.
- Ignoring computational cost.

10.9 Knowledge Check

1. How does boosting differ from bagging?
2. What does the learning rate control?
3. Why can boosting overfit?
4. What is stacking?

10.10 Real-World Case Study

10.10.1 Benchmarking Tabular Models

Conduct a transparent benchmark on a public health or education dataset.

10.11 Professor's Challenge

After testing 100 model configurations, one improves AUC by 0.003. Is this evidence of a better model?

10.12 Research Extension

Design a rigorous benchmarking study with repeated or nested validation and uncertainty around performance differences.

10.13 Reproducibility Check

Preserve all candidate models, search spaces, folds, seeds, runtimes, and selection rules.

10.14 Key Takeaways

- Ensembles can substantially improve tabular prediction.
- Fair benchmarking requires identical evaluation conditions.
- Small gains must be interpreted in context.

Part IV

Part IV — Unsupervised Learning

11 Clustering

11.1 Learning Objectives

- Explain the purpose and limitations of clustering.
- Apply K-means, hierarchical clustering, and DBSCAN.
- Select and interpret distance measures.
- Evaluate cluster stability and usefulness.
- Avoid treating discovered clusters as objective natural categories.

11.2 Why This Matters

Clustering can reveal structure in unlabeled data, but the resulting groups depend heavily on representation, scaling, distance, and algorithmic assumptions.

11.3 Real-World Problem

Can institutions, patients, or students be grouped into meaningful profiles without predefined labels?

11.4 Foundations

11.4.1 What Is a Cluster?

11.4.2 Distance and Similarity

11.4.3 Euclidean Distance

11.4.4 Feature Scaling

11.5 K-Means

11.5.1 Centroids

11.5.2 Objective Function

$$J = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

Choosing K

11.6 Hierarchical Clustering

11.6.1 Linkage

11.6.2 Dendrograms

11.7 DBSCAN

11.7.1 Density

11.7.2 Epsilon and Minimum Samples

11.7.3 Noise Points

11.8 Cluster Evaluation

11.8.1 Inertia

11.8.2 Silhouette Score

11.8.3 Stability

11.8.4 External and Substantive Validation

11.9 Hands-On Practice

Cluster institutions or health profiles using multiple algorithms and compare solutions.

11.10 Common Mistakes

- Clustering unscaled variables.
- Treating an elbow plot as definitive.
- Naming clusters without substantive validation.
- Assuming clusters are stable across samples.

11.11 Knowledge Check

1. What objective does K-means minimize?
2. Why does scaling matter?
3. How does DBSCAN differ from K-means?
4. What does silhouette score measure?

11.12 Real-World Case Study

11.12.1 Institutional Profiles

Cluster higher-education institutions using public IPEDS or College Scorecard features.

11.13 Professor's Challenge

Two clustering algorithms produce very different groups. Which result is “correct”?

11.14 Research Extension

Assess cluster stability across years, samples, or alternative feature representations.

11.15 Reproducibility Check

Document scaling, distance metrics, algorithm parameters, feature selection, and stability analysis.

11.16 Key Takeaways

- Clusters are model-dependent representations of structure.
- Validation must go beyond visual appeal.
- Substantive interpretation is essential.

12 Dimensionality Reduction

12.1 Learning Objectives

- Explain why dimensionality reduction is useful.
- Derive the intuition behind PCA.
- Interpret explained variance and component loadings.
- Distinguish PCA, factor analysis, t-SNE, and UMAP.
- Recognize when low-dimensional visualizations can mislead.

12.2 Why This Matters

High-dimensional data can be redundant, noisy, and difficult to visualize. Dimensionality reduction creates compact representations while introducing important interpretive tradeoffs.

12.3 Real-World Problem

A dataset contains dozens of correlated indicators. Can a smaller representation preserve the dominant structure?

12.4 PCA Foundations

12.4.1 Variance and Covariance

12.4.2 Eigenvalues and Eigenvectors

12.4.3 Principal Components

12.4.4 Explained Variance

12.4.5 Loadings and Scores

12.4.6 Scaling Before PCA

12.5 Related Methods

12.5.1 Factor Analysis

12.5.2 t-SNE

12.5.3 UMAP

12.5.4 Visualization Versus Modeling

12.6 Python Implementation

12.6.1 StandardScaler

12.6.2 PCA

12.6.3 Explained Variance Plots

12.6.4 Component Interpretation

12.7 Hands-On Practice

Reduce a multivariate health or education dataset and compare models before and after reduction.

12.8 Common Mistakes

- Applying PCA without scaling when scales differ.
- Interpreting components as causal constructs.
- Using t-SNE coordinates as ordinary quantitative variables.
- Choosing dimensions only because a plot looks attractive.

12.9 Knowledge Check

1. What does the first principal component maximize?
2. What is explained variance?
3. How does PCA differ from factor analysis?
4. Why should t-SNE be interpreted cautiously?

12.10 Real-World Case Study

12.10.1 Reducing Correlated Institutional Indicators

Use PCA to summarize institutional characteristics and examine downstream predictive performance.

12.11 Professor's Challenge

PCA improves computational efficiency but makes stakeholder interpretation harder. Should it be retained?

12.12 Research Extension

Evaluate whether latent representations are stable across populations or time.

12.13 Reproducibility Check

Document preprocessing, component-selection criteria, random seeds for stochastic methods, and interpretation rules.

12.14 Key Takeaways

- Dimensionality reduction trades original-variable interpretability for compact representation.
- PCA is grounded in variance and linear algebra.
- Visualization embeddings require cautious interpretation.

Part V

Part V — Modern Machine Learning

13 Neural Networks and Deep Learning

13.1 Learning Objectives

- Explain neurons, layers, activations, and loss functions.
- Describe forward propagation and backpropagation.
- Explain gradient-based optimization.
- Build a small neural network.
- Diagnose overfitting and apply regularization.

13.2 Why This Matters

Deep learning powers many modern AI systems and provides a foundation for understanding computer vision, NLP, and foundation models.

13.3 Real-World Problem

When does a neural network offer enough representational advantage to justify greater complexity than classical ML?

13.4 Mathematical Foundations

13.4.1 The Artificial Neuron

$$z = w^T x + b$$

Activation Functions ### Network Layers ### Loss Functions ### Gradient Descent

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t)$$

Backpropagation

13.5 Deep Learning Practice

13.5.1 Training and Validation

13.5.2 Epochs and Batches

13.5.3 Learning Rates

13.5.4 Regularization

13.5.5 Dropout

13.5.6 Early Stopping

13.6 Python Implementation

13.6.1 Building a Small Network

13.6.2 Training Curves

13.6.3 Evaluating Generalization

13.7 Hands-On Practice

Train a small neural network and compare it with a classical baseline.

13.8 Common Mistakes

- Using deep learning because it is fashionable.
- Ignoring baseline models.
- Evaluating repeatedly on the test set.
- Failing to monitor validation loss.

13.9 Knowledge Check

1. What does an activation function do?
2. What is backpropagation?
3. What does the learning rate control?
4. Why can early stopping improve generalization?

13.10 Real-World Case Study

13.10.1 Classical ML Versus Neural Networks

Compare model families on a structured dataset and justify the preferred model.

13.11 Professor's Challenge

A neural network improves accuracy by 0.5% but requires 50 times more computation. Is it better?

13.12 Research Extension

Investigate architecture, regularization, or transferability under a rigorous experimental design.

13.13 Reproducibility Check

Record architecture, initialization, seeds, hardware, software versions, epochs, and stopping criteria.

13.14 Key Takeaways

- Neural networks learn layered representations through optimization.
- Deep learning is powerful but not automatically superior.
- Reproducible training requires unusually careful documentation.

14 Natural Language Processing

14.1 Learning Objectives

- Explain how text can be represented computationally.
- Build text-classification pipelines.
- Compare bag-of-words, TF-IDF, and embeddings.
- Explain the intuition behind transformers.
- Evaluate NLP models responsibly.

14.2 Why This Matters

A large share of human knowledge is stored as text. NLP enables machine learning with documents, responses, reports, and other unstructured language data.

14.3 Real-World Problem

Can open-ended student responses or health narratives be analyzed reliably without reducing nuanced language to misleading labels?

14.4 Text Representation

14.4.1 Tokenization

14.4.2 Bag of Words

14.4.3 TF-IDF

14.4.4 N-Grams

14.4.5 Word and Document Embeddings

14.5 Modeling Text

14.5.1 Naive Bayes

14.5.2 Logistic Regression

14.5.3 Linear SVM

14.5.4 Neural Representations

14.6 Transformers

14.6.1 Attention

14.6.2 Contextual Embeddings

14.6.3 Pretrained Language Models

14.6.4 Fine-Tuning and Retrieval

14.7 Evaluation

14.7.1 Classification Metrics

14.7.2 Data Leakage in Text

14.7.3 Domain Shift

14.7.4 Human Evaluation

14.8 Hands-On Practice

85

Build a reproducible text classifier and compare sparse and embedding-based representations.

14.9 Common Mistakes

- Leakage from duplicated or related documents.
- Random splitting when time or source matters.
- Ignoring class and language imbalance.
- Treating model-generated labels as ground truth.

14.10 Knowledge Check

1. What does TF-IDF represent?
2. How are contextual embeddings different from bag-of-words features?
3. What problem does attention address?
4. Why can random splitting inflate NLP performance?

14.11 Real-World Case Study

14.11.1 Analyzing Educational Text

Develop a research-safe NLP workflow using a publicly available text corpus.

14.12 Professor's Challenge

An automated text classifier agrees with human coders 85% of the time. Is that sufficient for research use?

14.13 Research Extension

Compare human and machine coding, domain transfer, or robustness across populations.

14.14 Reproducibility Check

Version corpora, document preprocessing, preserve split logic, and record model/checkpoint identifiers.

14.15 Key Takeaways

- Text representation strongly shapes NLP performance.
- Transformers provide contextual representations but introduce new reproducibility challenges.
- Human validation remains important.

15 Generative AI and Foundation Models

15.1 Learning Objectives

- Explain how generative models differ from conventional predictive models.
- Describe the transformer and next-token prediction at a conceptual level.
- Explain prompting, embeddings, retrieval, and fine-tuning.
- Evaluate generative systems beyond anecdotal examples.
- Identify risks involving hallucination, bias, privacy, and reproducibility.

15.2 Why This Matters

Foundation models have changed how data scientists write code, analyze text, build interfaces, and design intelligent systems. Their outputs remain probabilistic and require systematic evaluation.

15.3 Real-World Problem

A university wants to use an LLM to provide research feedback to students. How should usefulness, factuality, consistency, bias, and educational consequences be evaluated?

15.4 Foundations

15.4.1 Generative Versus Discriminative Modeling

15.4.2 Language Modeling

15.4.3 Tokens and Context

15.4.4 Transformers and Attention

15.4.5 Next-Token Prediction

15.4.6 Foundation Models

15.5 Working With Foundation Models

15.5.1 Prompting

15.5.2 Structured Outputs

15.5.3 Embeddings

15.5.4 Retrieval-Augmented Generation

15.5.5 Fine-Tuning

15.5.6 Tool Use and Agents

15.6 Evaluation

15.6.1 Factuality

15.6.2 Reliability

15.6.3 Human Evaluation

15.6.4 Task-Based Evaluation

15.6.5 Reproducibility

15.6.6 Cost and Latency

15.7 Responsible Use

15.7.1 Hallucination

15.7.2 Bias

15.7.3 Privacy

15.9 Common Mistakes

- Treating fluent output as factual output.
- Reporting cherry-picked prompts.
- Failing to preserve model/version information.
- Using sensitive data without appropriate safeguards.

15.10 Knowledge Check

1. What is next-token prediction?
2. What is retrieval-augmented generation?
3. Why is generative-AI evaluation difficult?
4. How can model updates affect reproducibility?

15.11 Real-World Case Study

15.11.1 AI-Assisted Research Feedback

Develop an experimental protocol for evaluating LLM-generated feedback.

15.12 Professor's Challenge

Two evaluators strongly prefer an LLM's answers, but factual-error rates are higher. Which result should matter more?

15.13 Research Extension

Develop a controlled study suitable for education, HCI, or applied AI research.

15.14 Reproducibility Check

Record model provider, model identifier, date, prompts, parameters, retrieval corpus, evaluation rubric, and outputs where permissible.

15.15 Key Takeaways

- Generative AI requires evaluation as a system, not just a model.
- Fluency and correctness are different.
- Reproducibility and responsible use require new documentation practices.

Part VI

Part VI — Reliable and Research-Ready Machine Learning

16 Model Evaluation, Validation, and Hyperparameter Optimization

16.1 Learning Objectives

- Distinguish training, validation, and test data.
- Apply cross-validation appropriately.
- Explain nested cross-validation.
- Tune hyperparameters without contaminating final evaluation.
- Quantify uncertainty in model performance.

16.2 Why This Matters

A sophisticated model evaluated incorrectly can produce less credible evidence than a simple model evaluated rigorously.

16.3 Real-World Problem

After trying dozens of models, an analyst reports only the best test-set result. Why is that result no longer a trustworthy estimate of generalization?

16.4 Evaluation Foundations

16.4.1 Generalization Error

16.4.2 Train, Validation, and Test Sets

16.4.3 K-Fold Cross-Validation

16.4.4 Stratification

16.4.5 Grouped and Time-Based Splits

16.4.6 Repeated Cross-Validation

16.4.7 Nested Cross-Validation

16.5 Hyperparameter Optimization

16.5.1 Grid Search

16.5.2 Randomized Search

16.5.3 Bayesian Optimization

16.5.4 Search Budgets

16.6 Comparing Models

16.6.1 Baselines

16.6.2 Performance Distributions

16.6.3 Confidence Intervals

16.6.4 Practical Versus Statistical Differences

16.6.5 External Validation

16.7 Learning Curves

16.7.1 Diagnosing Bias

16.7.2 Diagnosing Variance

96

16.8 Hands-On Practice

Compare several models under identical repeated cross-validation and preserve an untouched final test set.

16.9 Common Mistakes

- Repeatedly checking test performance.
- Selecting folds that violate the data structure.
- Reporting only the winning configuration.
- Ignoring uncertainty around metrics.

16.10 Knowledge Check

1. What is the purpose of a validation set?
2. Why use nested cross-validation?
3. When is random K-fold inappropriate?
4. Why should model comparisons include uncertainty?

16.11 Real-World Case Study

16.11.1 A Reproducible Model Benchmark

Benchmark models on a public dataset using a preregistered evaluation protocol.

16.12 Professor's Challenge

Model A's mean AUC is 0.842 and Model B's is 0.845. Is Model B meaningfully better?

16.13 Research Extension

Design a benchmarking paper with robust validation, uncertainty, and external replication.

16.14 Reproducibility Check

Freeze split logic, folds, search spaces, metrics, seeds, and the final evaluation protocol before inspecting final results.

16.15 Key Takeaways

- Evaluation design determines the credibility of performance claims.
- The test set should approximate genuinely unseen data.
- Hyperparameter optimization must occur inside validation.

17 Explainable, Fair, and Responsible Machine Learning

17.1 Learning Objectives

- Distinguish global and local explanations.
- Apply feature-importance and explanation methods cautiously.
- Evaluate calibration and subgroup performance.
- Explain common fairness criteria and their tradeoffs.
- Conduct a model-risk assessment appropriate to the application.

17.2 Why This Matters

A model can be accurate overall while failing important populations, providing misleading explanations, or being inappropriate for the decision in which it is embedded.

17.3 Real-World Problem

A health model performs well overall but systematically underpredicts risk for one subgroup. Should it be deployed?

17.4 Explainability

17.4.1 Model Transparency

17.4.2 Global Versus Local Explanations

17.4.3 Permutation Importance

17.4.4 Partial Dependence

17.4.5 SHAP

17.4.6 Limits of Post-Hoc Explanation

17.5 Calibration and Reliability

17.5.1 Predicted Probability Versus Observed Frequency

17.5.2 Calibration Curves

17.5.3 Brier Score

17.6 Fairness

17.6.1 Subgroup Performance

17.6.2 Demographic Parity

17.6.3 Equal Opportunity

17.6.4 Equalized Odds

17.6.5 Calibration Across Groups

17.6.6 Fairness Tradeoffs

17.7 Responsible ML

17.7.1 Intended Use

17.7.2 Misuse

17.7.3 Human Oversight

17.7.4 Documentation

17.7.5 Model Cards

17.7.6 Monitoring and Drift

17.9 Common Mistakes

- Treating SHAP values as causal effects.
- Reporting only aggregate performance.
- Assuming removing protected variables removes bias.
- Choosing a fairness metric without considering context.

17.10 Knowledge Check

1. How do global and local explanations differ?
2. What is calibration?
3. Why can fairness criteria conflict?
4. Why does removing a protected attribute not guarantee fairness?

17.11 Real-World Case Study

17.11.1 Responsible Prediction in Health or Education

Conduct a structured model audit and document intended use and limitations.

17.12 Professor's Challenge

The most accurate model has the largest subgroup disparity. What evidence and stakeholder input are needed to decide what to do?

17.13 Research Extension

Develop a fairness, calibration, or explainability study with substantive domain relevance.

17.14 Reproducibility Check

Document group definitions, metrics, thresholds, explanation settings, software versions, and all audit decisions.

17.15 Key Takeaways

- Accuracy is necessary but not sufficient for responsible ML.
- Explanations are model-dependent and not automatically causal.
- Fairness requires explicit normative and technical choices.

18 Reproducible Machine Learning and Research Communication

18.1 Learning Objectives

- Build a reproducible ML project from raw data to publication.
- Use Git, GitHub, Quarto, and environment specifications.
- Document data and model provenance.
- Archive scholarly releases with persistent identifiers.
- Translate a machine learning analysis into research communication.

18.2 Why This Matters

A result that cannot be traced, audited, or reproduced is difficult to trust and difficult to build upon.

18.3 Real-World Problem

Six months after publication, can another researcher reproduce every table, figure, metric, and modeling decision in your study?

18.4 Reproducible Project Design

18.4.1 Project Structure

18.4.2 Raw Versus Derived Data

18.4.3 Scripts and Notebooks

18.4.4 Quarto

18.4.5 Git and GitHub

18.4.6 Environments and Dependencies

18.4.7 Randomness and Seeds

18.5 Data Provenance

18.5.1 Data Sources

18.5.2 Data Dictionaries

18.5.3 Licensing

18.5.4 Versioning

18.5.5 Sensitive Data

18.6 Model Provenance

18.6.1 Training Configuration

18.6.2 Hyperparameters

18.6.3 Evaluation Protocol

18.6.4 Artifacts and Model Cards

18.7 Scholarly Communication

18.7.1 Tables and Figures

18.7.2 Methods Reporting

106

18.7.3 Limitations

18.7.4 Repositories

18.7.5 CITATION.cff

18.10 Common Mistakes

- Manually editing outputs.
- Losing track of data versions.
- Failing to capture dependencies.
- Publishing only a notebook without execution instructions.
- Confusing an open repository with an archived scholarly release.

18.11 Knowledge Check

1. What is computational reproducibility?
2. Why separate raw and derived data?
3. What role does a software environment play?
4. Why create tagged releases?
5. What does a DOI contribute?

18.12 Real-World Case Study

18.12.1 From Notebook to Citable Research Artifact

Transform a chapter project into a documented, versioned, archived research output.

18.13 Professor's Challenge

A paper's repository contains code but the analysis no longer runs. Is the research reproducible?

18.14 Research Extension

Develop a replication or reproducibility study around a published machine learning analysis.

18.15 Reproducibility Check

Clone the project into a clean location and reproduce the complete analysis using only documented instructions.

18.16 Key Takeaways

- Reproducibility is a design principle, not a final checklist.
- Version control and environment capture make analytical decisions auditable.
- Scholarly archiving turns computational work into a persistent research artifact.

References